Detecting Abnormal Breathing Patterns using 1D CNNs

AI for Healthcare

Nikhil Rao
Email - cnikhilrao2006@gmail.com
March 2, 2026

Project Overview

AI is probably the most important thing humanity has ever worked on. I think of it as something more profound than electricity or fire.

~ Sundar Pichai, CEO of Google and Alphabet

With the boom of AI, Data has become a core resource for computer science. Without any doubt, Healthcare sector has emerged as one of the major producer of this resource. Every day tons and tons of data is produced be it by IoMT/IoHT devices, wearables like watches, or sensors at various hospitals. This project aims to leverage one such type of data, i.e time series data produced by SPO2, Nasal Airflow and Thoracic movement sensors, to non-invasively predict events like Apnea (complete airway closure) and Hypopnea (partial airway restriction) with the power of 1D CNNs.

The above mentioned data points of 5 different individuals over an 8-hour long sleep cycle is taken in, preprocessed, parsed into usable datasets, and then used to train models.

Signal Preprocessing and Visualization

This phase entails the rigorous removal of “un-natural” data units from the raw data. For example- while a person is wearing an SPO2 monitor on one of their fingers, the device slips off the finger and falls down. Now this would cause the SPO2 readings to go from say 98% to 0% for a couple seconds. We avoid using this particular data point for training and hence apply a few filters to recognize such events, set their values to np.nan and then interpolate the values. The same principle goes (with different technique) when a person’s breathing go to 10Hz from a normal range of 0.17Hz - 0.4Hz, implying that the person breadths in and out 10 times in one second, but we know that this is probably sensor error and clean that signal.

We must also align the data points as the provided sensors measure data at different rates. Hence we align the 4Hz SPO2 signal with the other 32Hz signals like Nasal Flow. Then we can plot these datapoints on the graph as shown below-

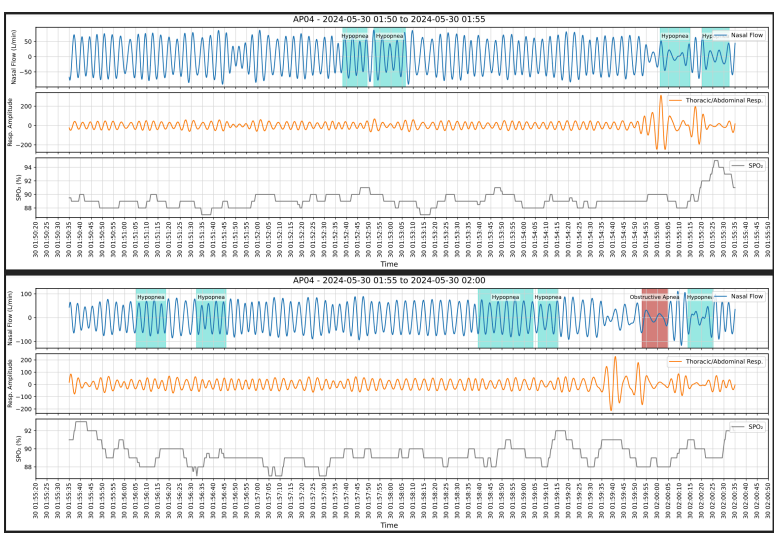
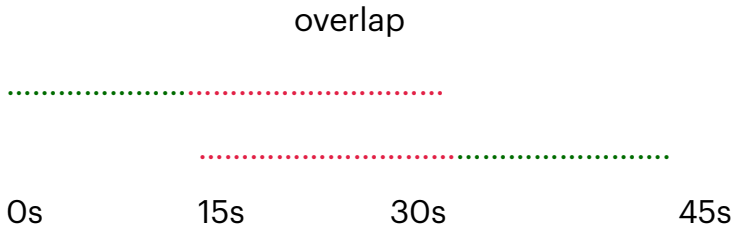


Fig-1 Visualization of a part of data of AP04

Dataset Architecture

In order to create suitable dataset for the models, we must create feature-label pairs. This entails classifying a certain period of data measured as “Normal”, “Obstructive Apnea“ or “Hypopnea”. We do so by using a 30 second window with a 50% overlap



We classify each 30 second window based on which condition exceeds 50% of the interval. For example, if a window has 70% “Normal” label and 30% “Hypopnea” label, we assign it as “Normal” and vise-versa.

We perform a strict inner join of time stamps, ensuring that we only take the time periods in account where all of our sensors were up and running.

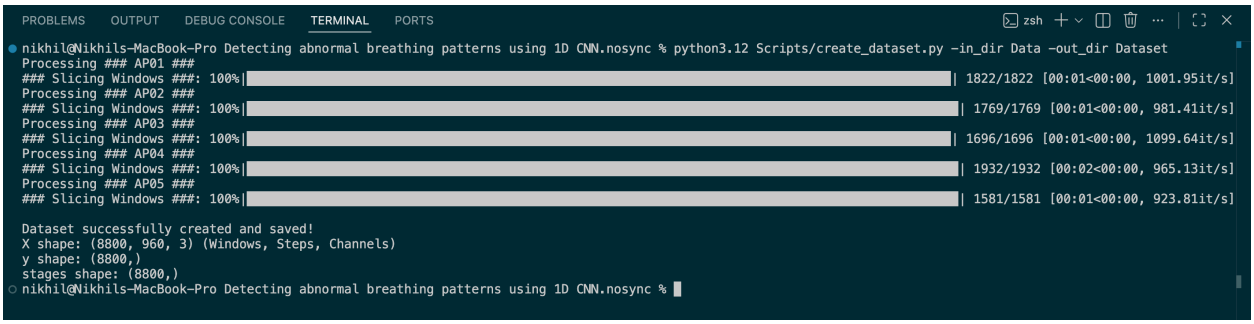


Fig-2 Terminal Output of the successful completion of the create_dataset.py script

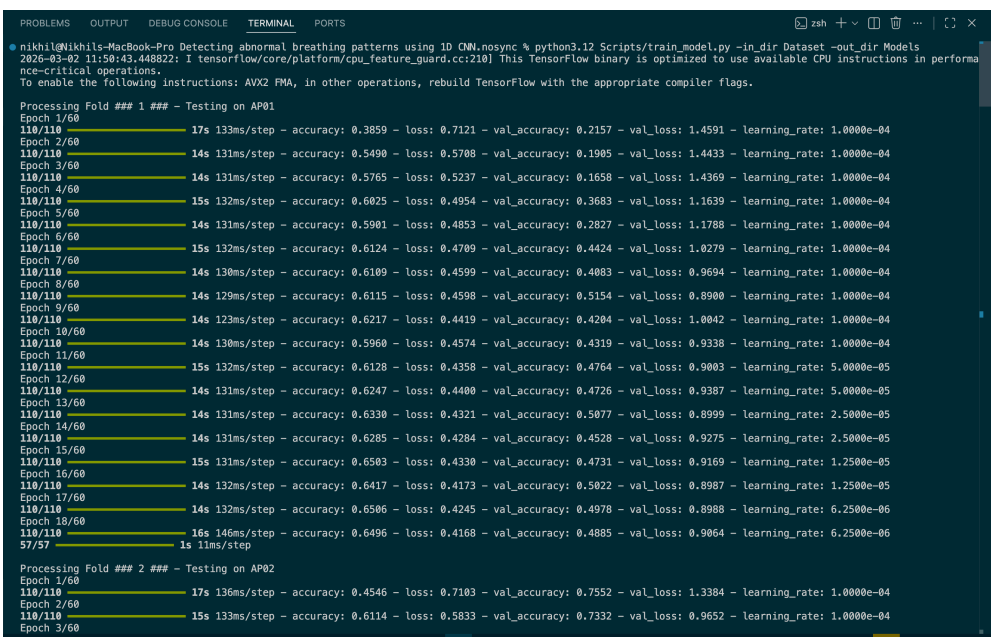
Model Architectures

1D CNN - The model leverages 1D Convolutional Neural Networks to perform spatial feature extraction from the 32Hz respiratory waveforms using Global Average Pooling and Regularization

Conv-LSTM (Hybrid) - To address the sequential nature of sleep events, I developed an experimental hybrid architecture. While the CNN blocks remain to capture local waveform shapes, the pooling layer is replaced with a **Bidirectional LSTM layer.** It tries to leverage its Temporal Context but still its accuracy and precision dips lower than the CNN probably due to its high data requirements.

The training phase was executed using a **Leave-One-Participant-Out Cross-Validation (LOPOCV)** strategy. This method entails training of Model on four participants and tested on a fifth, completely unseen individual. This process is rotated five times to ensure the model generalizes across all the available data.

To optimize for high precision and handle the extreme class imbalance (where "Normal" windows outnumber “Obstructive Apnea" events by almost 10 to 1), I implemented a Weighted Loss Function and used a small learning rate of 0.0001 which yielded significantly better precision for less common events than a learning rate of 0.0005 or 0.001, however at the end the over all matrices dipped cause of the gigantic “Normal” event section experienced a dip in accuracy and precision.



```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    zsh

nikhil@Nikhils-MacBook-Pro Detecting abnormal breathing patterns using 1D CNN.nosync % python3.12 Scripts/train_model.py -in_dir Dataset -out_dir Models
2026-03-02 11:50:43.448822: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performa
nce-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

Processing Fold ### 1 ### - Testing on AP01
Epoch 1/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 17s 133ms/step - accuracy: 0.3859 - loss: 0.7121 - val_accuracy: 0.2157 - val_loss: 1.4591 - learning_rate: 1.0000e-04
Epoch 2/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 131ms/step - accuracy: 0.5490 - loss: 0.5708 - val_accuracy: 0.1905 - val_loss: 1.4433 - learning_rate: 1.0000e-04
Epoch 3/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 131ms/step - accuracy: 0.5765 - loss: 0.5237 - val_accuracy: 0.1658 - val_loss: 1.4369 - learning_rate: 1.0000e-04
Epoch 4/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 15s 132ms/step - accuracy: 0.6025 - loss: 0.4954 - val_accuracy: 0.3683 - val_loss: 1.1639 - learning_rate: 1.0000e-04
Epoch 5/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 131ms/step - accuracy: 0.5901 - loss: 0.4853 - val_accuracy: 0.2827 - val_loss: 1.1788 - learning_rate: 1.0000e-04
Epoch 6/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 15s 132ms/step - accuracy: 0.6124 - loss: 0.4709 - val_accuracy: 0.4424 - val_loss: 1.0279 - learning_rate: 1.0000e-04
Epoch 7/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 130ms/step - accuracy: 0.6109 - loss: 0.4599 - val_accuracy: 0.4083 - val_loss: 0.9694 - learning_rate: 1.0000e-04
Epoch 8/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 129ms/step - accuracy: 0.6115 - loss: 0.4598 - val_accuracy: 0.5154 - val_loss: 0.8900 - learning_rate: 1.0000e-04
Epoch 9/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 123ms/step - accuracy: 0.6217 - loss: 0.4419 - val_accuracy: 0.4204 - val_loss: 1.0042 - learning_rate: 1.0000e-04
Epoch 10/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 130ms/step - accuracy: 0.5960 - loss: 0.4574 - val_accuracy: 0.4319 - val_loss: 0.9338 - learning_rate: 1.0000e-04
Epoch 11/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 15s 132ms/step - accuracy: 0.6128 - loss: 0.4358 - val_accuracy: 0.4764 - val_loss: 0.9003 - learning_rate: 5.0000e-05
Epoch 12/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 131ms/step - accuracy: 0.6247 - loss: 0.4400 - val_accuracy: 0.4726 - val_loss: 0.9387 - learning_rate: 5.0000e-05
Epoch 13/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 131ms/step - accuracy: 0.6330 - loss: 0.4321 - val_accuracy: 0.5077 - val_loss: 0.8999 - learning_rate: 2.5000e-05
Epoch 14/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 131ms/step - accuracy: 0.6285 - loss: 0.4284 - val_accuracy: 0.4528 - val_loss: 0.9275 - learning_rate: 2.5000e-05
Epoch 15/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 15s 131ms/step - accuracy: 0.6503 - loss: 0.4330 - val_accuracy: 0.4731 - val_loss: 0.9169 - learning_rate: 1.2500e-05
Epoch 16/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 132ms/step - accuracy: 0.6417 - loss: 0.4173 - val_accuracy: 0.5022 - val_loss: 0.8987 - learning_rate: 1.2500e-05
Epoch 17/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 14s 132ms/step - accuracy: 0.6506 - loss: 0.4245 - val_accuracy: 0.4978 - val_loss: 0.8988 - learning_rate: 6.2500e-06
Epoch 18/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 16s 146ms/step - accuracy: 0.6496 - loss: 0.4168 - val_accuracy: 0.4885 - val_loss: 0.9064 - learning_rate: 6.2500e-06
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step

Processing Fold ### 2 ### - Testing on AP02
Epoch 1/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 17s 136ms/step - accuracy: 0.4546 - loss: 0.7103 - val_accuracy: 0.7552 - val_loss: 1.3384 - learning_rate: 1.0000e-04
Epoch 2/60
110/110 ━━━━━━━━━━━━━━━━━━━━ 15s 133ms/step - accuracy: 0.6114 - loss: 0.5833 - val_accuracy: 0.7332 - val_loss: 0.9652 - learning_rate: 1.0000e-04
Epoch 3/60
```

Fig-3 Terminal Output of the Model training

Results

The tuned 1D CNN achieved a global **Precision of ~84%**. While the overall accuracy hovered near 51%, the model excelled at sensitivity—successfully catching nearly 40% more true apnea events than the baseline model. Conv-LSTM hybrid, while theoretically superior, struggled with the limited dataset size.

	precision	recall	f1-score	support
Body event	0.00	0.00	0.00	3
Hypopnea	0.08	0.49	0.14	593
Mixed Apnea	0.00	0.00	0.00	2
Normal	0.91	0.52	0.66	8038
Obstructive Apnea	0.02	0.07	0.03	164
accuracy			0.51	8800
macro avg	0.20	0.22	0.17	8800
weighted avg	0.84	0.51	0.61	8800

Fig-4 Classification Report of the 1D CNN model

Conclusion and Learnings

I got hands on experience dealing with raw IoMT/IoHT sensor data, how to clean that data, how to visualize that data, how to turn that data into a dataset useful for ML and training CNN models for time series data with LOPOCV. This project provided a deep dive into the "data-first" reality of AI for Healthcare. I learned that in medical signal processing, the cleaning and filtering phase (Phase 1) is just as critical as the neural network architecture itself.

*****